Figure 1 · End-to-end edge inference architecture — normal vs offline flows and failure modes

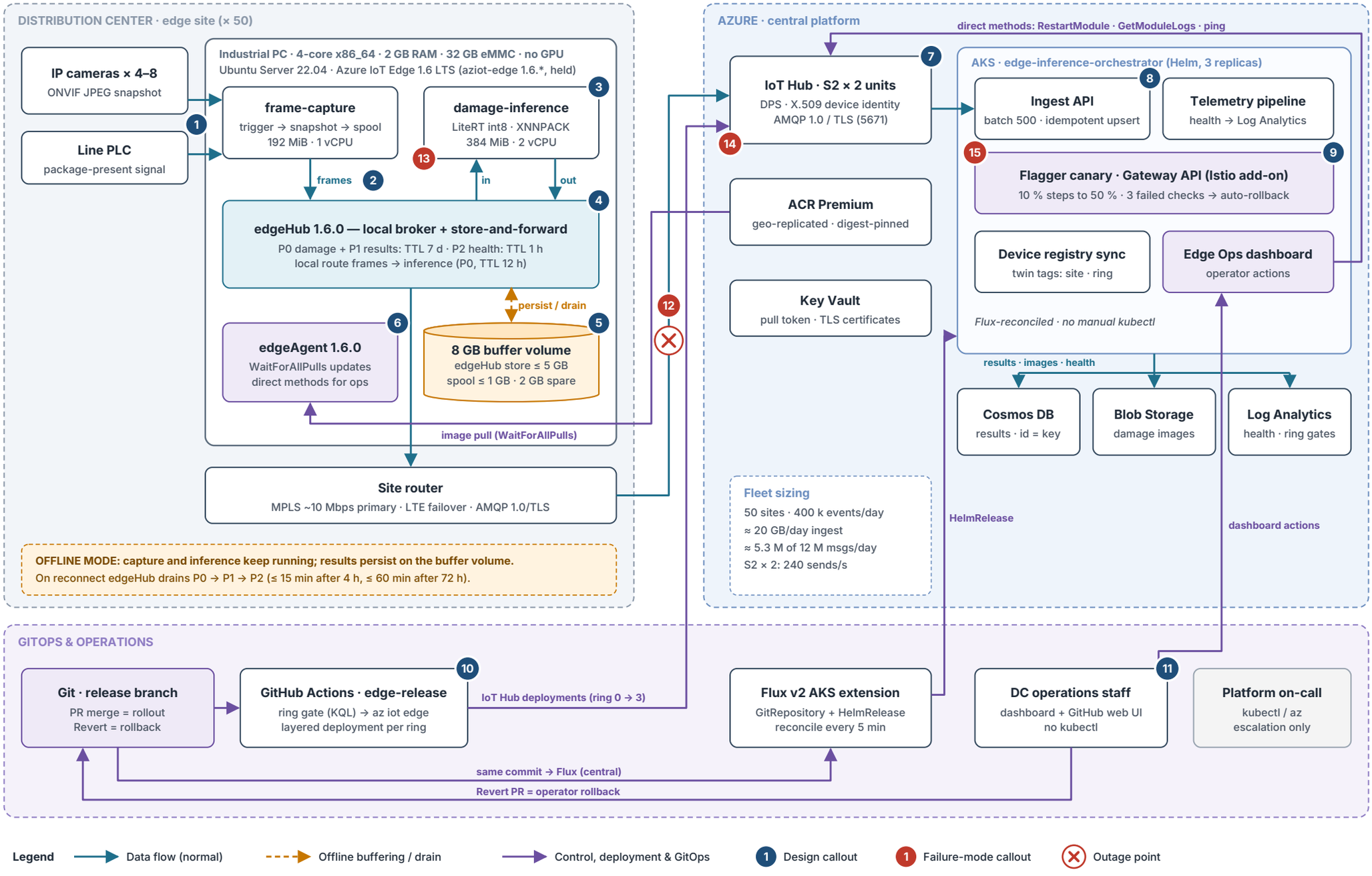
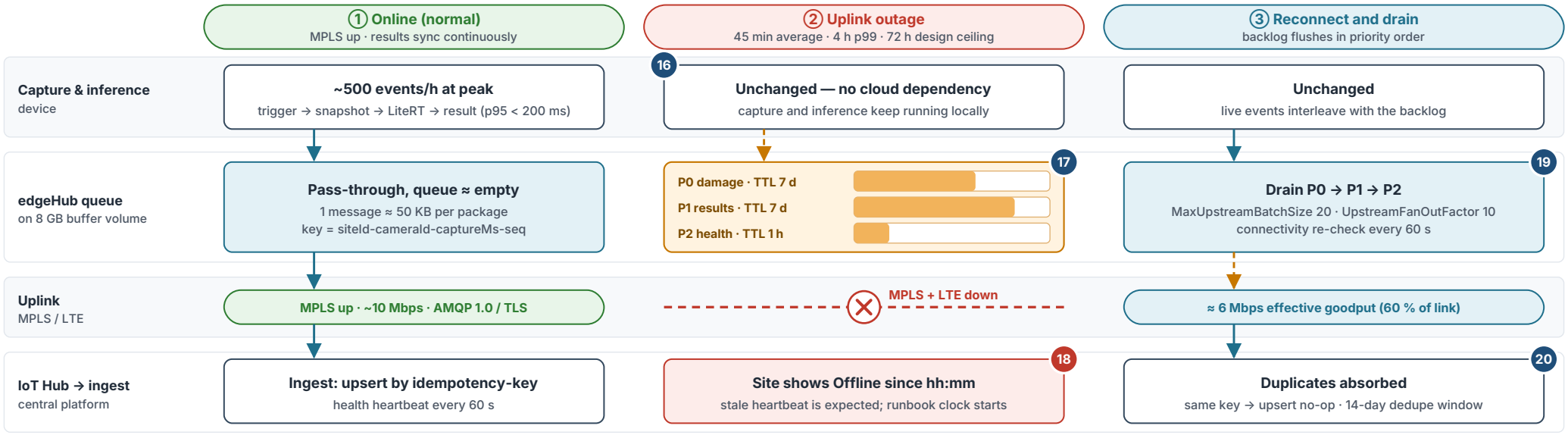


Figure 1. Each of the 50 sites runs event-triggered capture and LiteRT int8 inference on IoT Edge 1.6 LTS, buffers prioritized results on a dedicated 8 GB volume and syncs over AMQP to IoT Hub, while Git drives both the Flux-managed AKS orchestrator and the ring-based IoT Hub deployments.

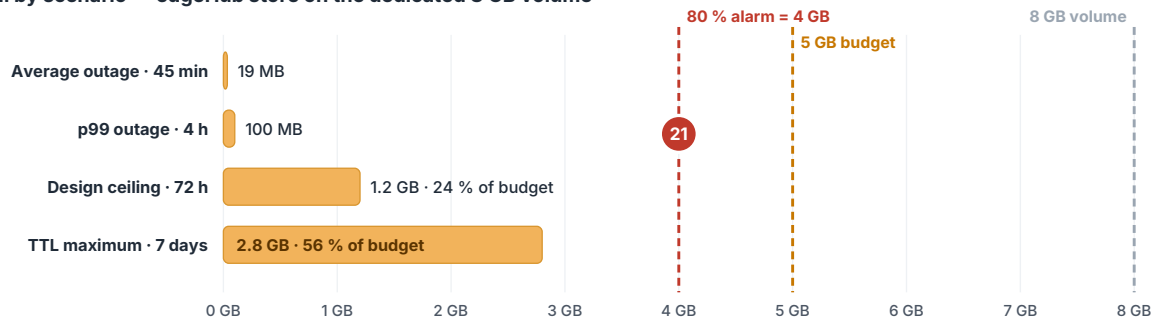
Callout key (→ section of Dennis_EdgeInference_Package)

1. Event-triggered capture (PLC + snapshot) saves CPU → §2, §3
2. Spool + local P0 route: no package missed during module restarts → §3, §6
3. LiteRT int8 + XNNPACK, 384 MiB cap, p95 < 200 ms gate → §1, §3
4. Route priority P0/P1/P2 + TTL 7 d / 7 d / 1 h; AMQP upstream → §3, §5
5. 8 GB volume: 5 GB queue + 1 GB spool + 2 GB headroom → §3
6. edgeAgent 1.6.0: WaitForAllPulls + direct methods for ops → §3, §7
7. IoT Hub S2 × 2: ≈ 5.3 M of 12 M metered msgs/day → §5
8. Deterministic idempotency key, upsert, 14-day dedupe → §5
9. Flagger canary on Gateway API (no Ingress-NGINX) → §4, §6
10. Ring rollout via layered IoT Hub deployments + KQL gate → §6
11. Operators: dashboard actions + PR Revert, never kubectl → §7
12. Uplink outage → offline mode (Figure 2) → §5
13. Module crash/OOM → restart policy + spool replay → §3, §7
14. Upstream rejection while online → buffer alarm → §5, §7
15. Bad release → automatic rollback (Flagger / ring gate) → §6

Figure 2 · Offline operation and reconnect drain — what happens during an uplink outage



Buffer fill by scenario — edgeHub store on the dedicated 8 GB volume



Peak-hour basis: 8,000 events ÷ 16 h × 50 KB = 25 MB/h. 72 h figure from the data package (400 MB/day × 3).
TTL (7 d) bounds growth at ~2.8 GB, so > 80 % means upstream rejection or an event storm, not an outage.
The remaining 3 GB of the volume holds the frame spool (≤ 1 GB) and RocksDB compaction headroom.

Drain time after reconnect (MPLS, ≈ 6 Mbps)

Scenario	Backlog	Drain	Target
45 min (avg)	19 MB	≈ 25 s	—
4 h (p99)	100 MB	≈ 2.2 min	**≤ 15 min**
72 h (ceiling)	1.2 GB	≈ 27 min	**≤ 60 min**

Per site ≈ 15 msg/s while draining (0.75 MB/s ÷ 50 KB).
10 sites draining at once ≈ 150 msg/s, below the
240 sends/s throttle of IoT Hub S2 × 2 units.
On LTE failover the drain is slower, but P0 damage
events still leave first; targets assume MPLS.

Legend — Data flow (normal) · Offline buffering / drain · 1 Design callout · 1 Failure-mode callout · Outage point

Figure 2. During an uplink outage the site keeps inspecting packages and queues results by priority, so even the 72-hour ceiling fills only 24 % of the 5 GB buffer budget and drains in about 27 minutes once MPLS returns.

Callout key (→ section of Dennis_EdgeInference_Package)

- 16 Capture + inference need no cloud: inspection never stops → §5
- 17 Damage events (P0) drain first; health expires after 1 h → §3, §5
- 18 Offline site status is expected; runbook clock starts → §7
- 19 Batch 20, fan-out 10, 60 s re-check; drain ≤ 15 / ≤ 60 min → §5
- 20 Replays and retries deduplicated by key (upsert no-op) → §5
- 21 > 80 % buffer = abnormal (TTL caps outage growth at ~2.8 GB) → §5, §7

Figure 3 · GitOps rollout, automatic rollback and operator-level recovery

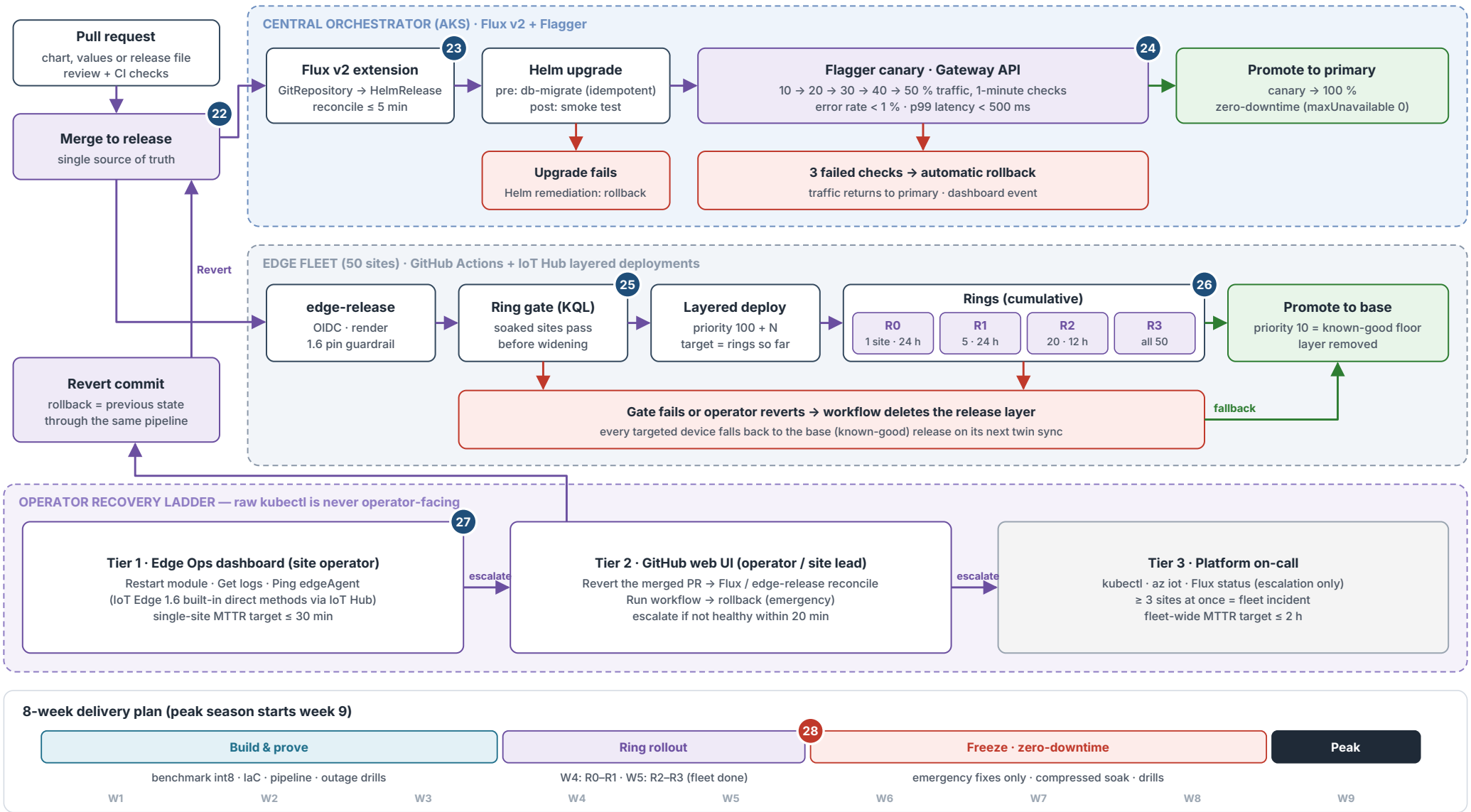


Figure 3. Every rollout and rollback is a Git commit: Flagger canaries the central orchestrator, IoT Hub layered deployments widen ring by ring behind a metric gate, and operators recover through dashboard actions and PR reverts, never raw kubectl.

Callout key (→ section of Dennis_EdgeInference_Package)

- 22 PR merge = rollout, Revert = rollback (single source of truth) → §6
- 23 Flux v2 reconcile ≤ 5 min; Helm hooks + remediation → §4, §6
- 24 Canary gates: error < 1 %, p99 < 500 ms, 3 failures → §6
- 25 Ring gate: p95, errors, restarts, sync lag, damage drift → §6
- 26 Base deployment = known-good floor; delete layer = rollback → §6
- 27 Recovery ladder with MTTR targets (30 min / 2 h) → §7
- 28 Freeze from week 6: fleet on release by end of week 5 → §6